

# Project Documentation Outline

**Course:** Design and Analysis of Algorithms

**Project Title:** Bogosort: A Comparative Analysis to the Question No One Needed

**Name:**

Abril, Rheigne Ysabel M., Alcantara, Daedan Ryu E., Anyayahan, Jerlyn P., Asilo, Sofhia Aubrey M., Bautista, Angelica Joy G.

**Date Submitted:** May 11, 2026

*BATANGAS STATE UNIVERSITY – ALANGILAN CAMPUS*

*COLLEGE OF INFORMATICS AND COMPUTING SCIENCES*

*COMPUTER SCIENCE*

# **I. Abstract**

Some questions are not in need of asking due to practicality, but serves as a cornerstone to pedagogy and development. Such is Bogosort, a highly inefficient joke algorithm that serves no practical purpose but to show what not to do in the spirit of effective algorithm development. In this analysis, the researchers observed and measured the significant time complexity and structure of bogosort against other existing esoteric algorithms to define which properties of the algorithm makes it inefficient and slow in performance. Specific parameters in the analysis and experimental cases include taking a sortable list of integers and applying algorithmic procedures until the elements are sorted – defined as having the  $n$ th element smaller than its  $n+1$ th element thus, consequent elements in ascending order. Test cases indicate a significant degree of inefficiency in bogosort compared to algorithms such as Thanos Sort and Stalin Sort wherein faster execution is exchanged with lossy performance. Meanwhile, Sleep Sort is able to achieve high data integrity but fails to be less effective than bogosort. It is with Bogobogosort where the recursive nature of the strange sorting algorithm exponentially advances bogosort in terms of inefficiency. Though not considered as the most inefficient, Bogosort is a pioneer in the realm of joke algorithms. Serving one purpose – to teach about efficacy in the spirit of the opposite. To show what not to do when designing an algorithm.

## II. Introduction

In the realm of logic and reason, there comes a shimmer of pure absurdity that guides the overarching body of knowledge from self-destruction. There goes the saying, “If Stupidity got us into this mess, then why can't it get us out?” Learning what not to do in such scenarios helps us realize the way towards the right path. Experiencing those mistakes builds our critical thinking skills in a way that makes us think about what went wrong, why it happened that way, and what we can do to avoid making the same mistake. Such is the case for bogosort, an algorithm born from a joke that shouldn't exist in the practical sense.

Bogosort – named after the term ‘Bogus’ meaning counterfeit, sham, fake, joke (Merriam-Webster, n.d) and also synonymous with the term ‘stupid sort’ and ‘monkey sort’ (Raymond, 1996, as cited in Gruber et al., 2007) (Holzer and Maurer, 2018) – is the forefront of the definition of pure absurdity and stupidity in the realm of algorithms. Being a highly inefficient sorting algorithm, it poses no practical and concrete solution to any real-world problems that exist in the modern day. Its existence only serves as pedagogy to illustrate distinct differences in theoretical and practical correctness. Just because an algorithm has run successfully, it does not mean that it will be the best solution. The conception of bogosort stemmed from a problem that no one needed an answer to: “What is an inefficient way to sort a series of values?”

In this project, the researchers are aimed to test and compare the inefficiency of bogosort compared against other impractical sorting algorithms made over time after the conception of

the bogosort (e.g. Sleep Sort, Stalin Sort, Thanos Sort) in terms of sorting a fixed length of values inside an array. Optimization is out the window. Just a simple parameter; find the slowest and least efficient way to sort a set of values. Any and all possible approaches are on the table to accomplish the task.

Defining the characteristics of each sorting algorithm – practical or not – and comparing their strengths and weaknesses can generate a sense of suitability on each algorithm tested. Furthermore, the project can test the pioneering ‘bogosort’ algorithm compared with newer esoteric algorithms to define the inefficiencies among the algorithms presented and how worse they are.

### **III. Problem Definition**

As opposed to conventional computational problems that require full on optimization, inefficiency is the target in this comparative analysis. Searching the most ineffective solving algorithm must have the correct procedures taken to theoretically sort a list of sortable elements. The given algorithm – though highly poor in performance – must still point towards a desired goal. Furthermore, the goal of the algorithm must be to take a list of integer elements and display the sorted version of that list. A sorted list is based on how many elements present, must have been arranged in order from lowest numerical value to highest numerical value. While correctness remains a requirement, the focus of the analysis depends on how inefficiently this objective is achieved.

With this in mind, this study intentionally tackles the problem of evaluating and comparing impractical sorting algorithms in a controlled experimental setting. Given identical input conditions, the project seeks to analyze how these algorithms differ in terms of execution behavior, computational effort, and consistency of completion. Measurable criteria such as execution time, number of iterations, and likelihood of termination are applied to establish a fair comparison. The goal is to determine which algorithm demonstrates the greatest inefficiency while still producing a correct output, emphasizing the limitations of correctness as the only measure of algorithmic quality and strengthening the importance of efficiency in algorithm design.

The input to the algorithm is an array of  $n$  integers, where  $n$  is a finite, fixed value for each test case. The array may contain duplicate values, and no assumption is made about the

initial arrangement of its contents. The array may arrive already sorted, in reverse order, or in any random configuration. When  $n = 0$ , the input is an empty list, which is considered trivially sorted and requires no further action from the algorithm. The output is a rearrangement of the same array in which every element at position  $i$  is less than or equal to the element in position  $i + 1$ , for all valid indices.

Bogosort operates on the generate and test paradigm: it repeatedly generates random permutations of the input array and checks whether the resulting arrangement is sorted, continuing this cycle until a sorted permutation is produced by chance. Two variants of this algorithm exist – a deterministic version that systematically generates and tests every possible permutation of the input until the sorted arrangement is found, and a randomized version that shuffles the array randomly on each iteration.

The randomized variant carries no upper bound on the number of steps required, as there is no guarantee that any given shuffle will yield a sorted result, making the worst-case time complexity theoretically unbounded. The best case, by contrast, occurs when the input array happens to already be sorted upon the algorithm's first check, requiring only a single pass of  $O(n)$  to verify.

The sorting problem itself is defined neutrally, and Bogosort does satisfy this definition eventually. The research objective, however, is to interrogate the nature of that 'eventually', which is to analyze and compare just how inefficiently Bogosort and other similarly absurd

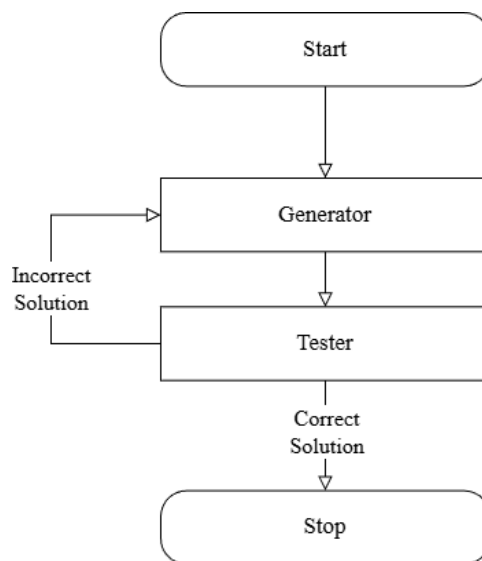
algorithms arrive at a correct answer, and what that inefficiency can teach about the boundaries of correctness versus practicality.

To maintain consistency across all trials, the study is conducted under several constraints and assumptions. Each test case has a fixed input size  $n$ , and each array is made up only of integer values that may include duplicates and may appear in any initial order. Despite the intentional emphasis on inefficiency, all algorithms are still expected to generate a correctly sorted output in ascending order. Measurable metrics such as execution time, number of iterations performed, and the probability of termination are used to assess performance. Randomized algorithms, including Bogosort, are assumed to have no fixed upper bound on runtime, while predictable variations systematically explore every possible permutation until a correct arrangement is found. Identical inputs are used to guarantee fairness in comparison, and external conditions like system load and hardware performance will remain constant.

## IV. Literature Review / Related Work

Sorting is one of the many fundamental actions that is the cornerstone of data management in computer science. Since the infancy of the discipline, many algorithms with various qualities, strengths, and weaknesses are created to accomplish the task (Ciupercă et al., 2023). However, the common notion with sorting algorithms is that the goal is to accomplish the most effective and fastest sorting process. Many conventional algorithms align with this notion such as Merge sort, Quick Sort, Bubble Sort and more.

In the spirit of humor and curiosity, some algorithms are developed to be intentionally terrible or awful. Many of these algorithms are mainly on the same Generate-and-Test paradigm in essence that they first generate a solution from the problem, then tests if the created solution would be on par with the goal, then checks if it would be the best possible solution (Sanchhaya Education Private Limited, 2021).



*Figure 1. Generate-and-Test Algorithm Paradigm*



This type of approach to sorting guarantees that the best possible solution is checked against all other solutions. One downside of this approach is that the time spent on generating all possible solutions then checking is costly especially on larger data sets. That weakness of the paradigm is essential in creating esoteric algorithms that are designed to be as inefficient and slow as possible (Barták, 1998, as cited in Delecluse & Schaus, 2024).. Another setback to the paradigm is thrashing. Thrashing is coined as the repeated failure of the algorithm due to a singular reason. Thrashing is caused by the repeated swapping of elements that in turn degrade the performance of the system (Denning, 1968, as cited in Ma et al., 2025). Algorithms such as bogosort exploit that weakness and ask how to worsen it.

Likewise stated earlier, bogosort is a generate-and-test algorithm that generates all possible solutions then tests them for the best one. An aspect unique about bogosort is that it is purely permutative, meaning that its solutions are different permutations or order of the values inside an array (Perchersky, 2023). The methodology behind bogosort is commonly described as throwing a deck of cards in the air and picking them randomly to check if the cards are now in order. That lack of technique makes the algorithm severely inefficient and slow in terms of time complexity. The algorithm's generation of permutation solutions is highly dependent on sheer randomness. There is no approach to minimize any inefficiency such as partial completion of the sort brought by a lucky shuffle. If the array is not completely sorted, it is then reset into the next permutation. The shuffling of elements in the array also possesses the disadvantage of redundant work wherein failed solutions are not detected or remembered onto the subsequent computation (Barták, 1998, as cited in Delecluse & Schaus, 2024). Therefore, a previously generated incorrect permutation can be

generated again based on probability (Ciupercă et al., 2023). Bogosort is so inefficient in its technique that it is noted that there is a chance that the algorithm may never end or find the correct sorting permutation because it uses purely random sorting (Lerma, 2014).

Conventional or effective sorting algorithms mainly use the generate-and-test paradigm due to their combinatorial nature. Though unlike naive algorithms such as bogosort, they add intelligent techniques such as backtracking to further optimize their execution.

Conversely, there are absurdist algorithms on the opposite end of the spectrum where all optimization is thrown off the window. These deliberately-naive algorithms are structured to have the most deficiency in any way, shape, or form possible. Such examples of algorithms can be humorous. For example, Stalin Sort. Also known as dropsort, Stalin Sort is a sorting algorithm that if it detects that an element is not ordered within the sequence, the algorithm pushes the element to a new array or ‘the gulag’ rather than sorting it (*Stalin Sort*, 2026). The algorithm repeats the sequence of steps until it can verify that the remaining elements in the sequence are ordered. Though Stalin Sort boasts an incredible time complexity of  $O(n)$ , the algorithm cannot be classified as an effective sorting algorithm due to the questionable integrity of data within the sequence. Stalin sort is commonly classified as a *lossy algorithm* as its fast performance coincides with a potential loss of data during execution (Morgan-Mar, 2013). There is no actual sorting mechanism taking place inside but rather just removing elements in one go until it could technically accomplish the task of presenting a ‘sorted’ array (Ivanchikov & Antsuto, 2025).

```
FUNCTION stalinSort(A : list OF sortable items)
  n := length(A)
```

```

bigger := 0
B SET empty list
FOR i := 0 TO n NOT inclusive DO
    IF A[i] >= bigger THEN
        bigger := A[i]
        B.push(A[i])
    END IF
END FOR
RETURN B
END FUNCTION

```

*Figure 2. Stalin Sort Pseudocode. Source: Ivanchikov & Antsuto, 2025*

The performance of Stalin sort is dependent on the data being implemented to the algorithm. In a study by Ivanchikov and Antsuto in 2025, they calculated that the best case scenario on the algorithm's time complexity was linear if the input set was already sorted before any implementation by the algorithm. In its worst possible case, the complexity would still be  $O(n)$ . However, the output set would return in high plausibility a single element – first element in the set – as the list would shrink as the algorithm traverses on. Stalin Sort prioritizes accomplishing the concept of order rather than keeping a complete set. In this case, though the elements are severely compromised, the algorithm achieves the objective of a sorted list (Ivanchikov & Antsuto, 2025).

Thanos Sort is one of the likes of Stalin Sort. It is also a humorous sorting algorithm that implements a list and orders its elements according to what classifies as a sorted array. In a general sense, the premise of Thanos Sort is that if the given array elements are not yet sorted, the algorithm will remove 50% of the array then check if the array is sorted. If not, it will continue to delete 50% of the remaining array until the array elements are sorted accordingly. Like Stalin Sort, it can qualify in the requirement that it gives back a sorted

version of the input array. However, it sets aside the concept of completeness in a sense that it would compromise the data given for the sake of accomplishing the goal. It was conceptualized in 2019 as an April Fool's Challenge by Mike Mirzayanov and since then added as an ineffective algorithm made by joke (*Mirzayanov, 2019*).

Since 2019, there have been some variations of the Thanos Sort that uses different designs according to requirements being set. Two of those variations use the Generate-and-Test paradigm and the Recursive design. In the Generate-and-Test algorithm, the Thanos Sort would remove half the number of elements in an array randomly until it satisfies the condition of the array being sorted.

```
FUNCTION thanosSortA (A: list OF sortable items)
  WHILE A is NOT sorted DO
    N := length(A)/2
    randomly remove N elements from A
  END WHILE
  RETURN A
END FUNCTION
```

*Figure 3. Thanos Sort Generate-and-Test Pseudocode. Adapted From: Chandraseta,*

*2021*

```
FUNCTION thanosSortB (A: list OF sortable items)
  IF A sorted DO
    size := length(A)
    RETURN size
  END IF
  middle := length(A)/2
  leftHalf := A[0, middle-1]
  rightHalf := A[middle, length-1]
  RETURN max(thanosSort(leftHalf), thanosSort(rightHalf))
END FUNCTION
```

*Figure 4. Thanos Sort Recursion Pseudocode. Adapted From: Mirzayanov, 2019*

Recursive Thanos Sort is more specified as it would return a subset of the array that would qualify being sorted. Its recursive nature means that the algorithm will split the array into two halves then independently split those halves until a sorted subset is found.

|                  |               | Generate-and-Test | Recursion     |
|------------------|---------------|-------------------|---------------|
| Time Complexity  | Best Case     | $O(n)$            | $O(n)$        |
|                  | Expected Case | $O(n!)$           | $O(n \log n)$ |
|                  | Worst Case    | $O(n)$            | $O(n \log n)$ |
| Space Complexity |               | $O(n)$            | $O(n)$        |

*Figure 5. Comparison of Time and Space Complexity of Thanos Sort versions*

The performance of each version varies due to their use of distinct algorithm designs. Though more straightforward, generate-and-test thanos sort is much more slower with an expected case of  $O(n!)$ . This is due to the generate-and-test's weakness of forcing through every iteration until a valid solution is met. Recursive Thanos Sort splits the array until it finds the subset that is sorted which speeds up the iteration exponentially.

Another example of an absurdist algorithm was designed to have a waiting time before the complete sorting be initiated called Sleep Sort.

```
function sleep_and_print(x):  
    sleep(x)  
    print x  
  
function sleep_sort(A):  
    for every x in list A:  
        create_a_new_thread(sleep_and_print(x))  
    add all the threads with the main thread
```

*Figure 6. Sleep Sort Pseudocode. Reprinted From: Kaushik, 2018*

Sleep Sort uses a concept called ‘threading’ wherein elements inside the array ‘sleeps’ for an interval according to the value of the element itself. Threading is a concept where the computer’s scheduler controls the flow in which parts of a process are being executed at a certain point within the whole sequence (Kaushik, 2018). In this case, the system threads on the sorting of each element and schedules when to complement each sort according to their value (Lamport 1979, as cited in Leiserson et al., 2022).

This algorithm, like others mentioned, is not a practical and viable option in real-world scenarios, it is being used to illustrate the process of multi-threading and how the scheduler in a computer works (Altcademy [Altcademy], 2023). Sleep Sort’s definitive time complexity is currently a debate, as many programmers argue that the algorithm is highly system-dependent due to the use of an environment scheduler rather than relying on predictable techniques.

According to Neil Kalman, a web developer and software engineer:

*Since Sleep() function and creating multiple threads is done internally by the OS using a priority queue (used for scheduling purposes). Hence inserting all the array*

*elements in the priority queue takes  $O(N \log N)$  time. Also the output is obtained only when all the threads are processed, i.e- when all the elements 'wakes' up. Since it takes  $O(arr[i])$  time to wake the  $i$ th array element's thread. So it will take a maximum of  $O(\max(input))$  for the largest element of the array to wake up. Thus the overall time complexity can be assumed as  $O(N \log N + \max(input))$ , where,  $N$  = number of elements in the input array, and  $input$  = input array elements (Kalman, 2018)*

Other developers claim that the time complexity of the algorithm would be truly undefined as the reliance on the computer's central processing unit would allow for extraneous variables like processing speed to cause the algorithm to become unpredictable (Shivananda, 2023).

Some inefficient algorithms are evolutions from the pioneering bogosort. Algorithms such as bogobogosort build upon the existing generate-and-test paradigm of bogosort and further amplifies it by recursively calling that bogosort into a subsection of the array. The algorithm must complete implementing bogosort in all subsections of the array to end. David Morgan-Mar (2013) as cited in Henney (2022) lists the procedure and pseudocode as:

1. Make a copy of the list of numbers.
2. Sort the first  $n-1$  elements of the copy using bogobogosort.
3. Check to see if the  $n$ th element of the sorted copy is greater than the highest element of the first  $n-1$  elements. If so, the copy is now sorted, else randomise the order of the elements of the copy and go to step 2.
4. Check to see if the copy is in the same order as the original list.

```

FUNCTION bogobogosort(A: list OF sortable items)
  n := LENGTH(A)

  IF n <= 1 THEN
    RETURN A
  END IF

  ACopy := A.COPY()

  WHILE TRUE DO
    ACopy := bogobogosort(ACopy[0 TO n-2])

    IF ACopy[n-1] > MAX(ACopy[0 TO n-2]) THEN
      IF ACopy == A THEN
        RETURN ACopy
      ELSE
        ACopy := RANDOMIZE(ACopy)
      END IF
    ELSE
      ACopy := RANDOMIZE(ACopy)
    END IF
  END WHILE
END FUNCTION

```

*Figure 7. Esoteric BogoBogoSort Pseudocode. Adapted From: Morgan-Mar, 2013*

There are various interpretations on the time complexity of the algorithm due to its structure and ridiculousness. A statement from Michael Rosulek, a professor of Electrical Engineering and Computer Science at the College of Engineering in Oregon State University (2006, as cited in Morgan-Mar, 2013) says that his analysis is that the expected time complexity of bogobogosort is  $O(n * (n!)^n)$  due to the combined time of sort checking, bogosorting, with the complexity of a  $\frac{1}{n!}$  probability of the array being sorted. According to his statement, the overall equation for the algorithm's complexity is:

$$BBS(n) = SC(n) + (1 - \frac{1}{n!})BBS(n)$$



In the given equation we let  $BBS(n)$  as the expected bogobogosort time while  $SC(n)$  is the expected time from sort checking during the implementation. Furthermore, his analysis of the  $SC(n)$  time complexity stands at:

$$SC(n) = O(n) + BBS(n - 1) + (1 - 1/n)(BBS(n - 1) + O(n))$$

In this equation, the  $SC(n)$  has an  $O(n)$  time complexity which means that the sort checking procedure must always make sure that the  $n$ th position is checked after sorting the  $n-1$ th elements. However, he found that if the  $n$ th element is not sorted even if the  $n-1$ th elements are, the bogobogosort will shuffle all elements – resetting the progress from the earlier recursion. This recursion reset sets the algorithm back into loops of wherein progress will be negated:

*Then substituting in for SC (skipping some simplifications - the  $(1-1/n)*O(n)$*

*factor gets swallowed by the first  $O(n)$ ):*

$$BBS(n) = O(n) + BBS(n-1) + (1-1/n) BBS(n-1) + (1-1/n!) BBS(n)$$

*// multiply by  $n!$*

$$n! * BBS(n) = O(n*n!) + n! ( BBS(n) + 2 BBS(n-1) ) - BBS(n) - (n-1)!*BBS(n-1)$$

*// rearrange*

$$BBS(n) = O(n*n!) + (2n! - (n-1)!)BBS(n-1)$$

$$= O(n*n!) + O(n!) * BBS(n-1)$$

$$= O(n*n!) + O(n*(n!)^2) + ... + O(n*(n!)^n)$$

$$= O(n*(n!)^n)$$

In this recursion loop, the permutation computation can be interpreted as:

$$T(k) = T(k - 1) + (k - 1)/k \times [ T(k - 1) + T(k) ]$$

Where for each failure after sorting  $T(k - 1)$  with a probability of  $(k - 1)/k$ , the cost would be another  $T(k - 1)$  with the same lengthy probability.

Another interpretation from Nick Collins, a PhD student of Portland State University (2006, as cited in Morgan-Mar, 2013) interpreted the bogobogosort in such that:

$$B(n) \leq n! \cdot (Copy(n) + Check(n) + Randomize(n))$$

This analysis of bogosort uses an upper-bound estimate instead of a strict equality that would find the expected time complexity. The built analysis is derived through a two-staged argument of establishing a structural recurrence relation over the operations of the algorithm and then expanding the recurrence into a closed product. Collin's analysis stated that the total cost of the bogobogosort is bounded at the maximum  $n!$  as it is the probability of getting the sorted sequence through random selection. The expected number of repetitions is applicable to the three sub-operations of the algorithm namely  $Copy(n)$ ,  $Check(n)$ , and  $Randomize(n)$ .

The  $Check(n)$  operation according to Collin would be a recursion call for each  $n-1$  element to see if the last element is the largest. The probability of the operation would be  $\frac{1}{n}$  for each recursion of  $n-1$  elements. The  $Copy(n)$  is then used to create a separate copy of the subsequence to be checked. If the recursion check fails to see a sorted subsequence, the  $Randomize(n)$  would be put in place. The analysis would be then expanded as:

$$Check(n) \leq n * (B(n - 1) + Max(2) + Randomize(n)) + Compare(n)$$

In turn, the  $Check(n)$  becomes the most time complexive sub-operation in the whole algorithm. Collins further added that  $Randomize(n)$  would be  $O(n)$  and he then expanded the  $Check(n)$  into a recurrence until the final computation is as follows:

$$B(n) = O(n! * n! * n! * n! * (n - 2)! * (n - 3)! * \dots * (k + 1)!)$$

Each  $(n - j)!$  are then bounded by  $n!$  until  $B(n)$  is simplified into  $O(n!^{n-k})$  for any fixed  $k$ .

Difference in the analysis of the two hugely lies on the approach made with equivalent trade-offs in certain aspects of the analysis. Rosulek's analysis used stricter mediums in the computation to find the time complexity of bogobogosort. The use of probabilities further strengthens the justification for the complexity. However, that precision could weaken the overall estimation of the algorithm. Furthermore, his analysis relies on assumptions that the randomness present in the algorithm would be uniform and independent. Rosulek's complex and probabilistic recurrence solving – though tight in estimation – might not characterize fully the extent of inefficiency. Collins on the other hand uses a deterministic approach in that using an upper bound can widen the picture of the estimate. Though his analysis is inherently looser than Rosulek's, the approach presents a simpler computation that presents a safe estimation of the algorithm's expected complexity. The interpretations made by Rosulek and Collins – though different in the approach and result – accurately tackles the structure of bogobogosort to create a clear picture of the algorithm's performance.

Across the surveyed esoteric sorting algorithms, a clear pattern emerges: inefficiency is not a single, uniform property but something that can be adjusted across multiple dimensions—time complexity, data integrity, memory usage, termination probability, and dependence on external systems. Bogosort and its recursive counterpart, Bogobogosort, illustrate purely computational inefficiency, relying on unbounded randomness and repeated resets that lead to factorial or even worse time complexities. In contrast, Stalin Sort and Thanos Sort achieve linear or near-linear performance by intentionally discarding elements, highlighting a direct trade-off between speed and completeness. Sleep Sort represents a different category altogether, where performance depends on system-level factors, calling into question the traditional notion of algorithmic complexity. Taken together, these algorithms are not meant for practical use; instead, they act as conceptual boundary cases that expose underlying assumptions in conventional sorting theory—such as guaranteed termination, data preservation, and deterministic execution time. Their appeal lies in how they push these assumptions to their limits, often in deliberately absurd ways.

## V. Proposed Solution / Algorithm Description

Bogosort is a notoriously inefficient sorting algorithm based on the "generate and test" paradigm; it works by repeatedly shuffling the entire list to generate a random permutation and checks again if it is still not sorted. Since Bogosort relies on chance, the algorithm may take much longer to sort and It is also possible that Bogosort may never sort the list at all. In this study, Bogosort is used as the core algorithm to examine how a sorting procedure operates without optimization mechanisms, being widely used for pedagogical purposes, humor, or to demonstrate an extreme algorithmic inefficiency, focusing on its structural behavior rather than performance efficiency nor practical application.

The algorithm does not employ structured design such as divide-and-conquer or greedy methods. It performs no state retention and does not utilize partially sorted information to improve future iterations. Each unsuccessful attempt resets the process entirely, making the approach purely iterative and dependent on randomness.

The research focuses on evaluating the algorithm's lack of optimization – specifically, how it executes without state retention or heuristic tracking.

The execution of the algorithm is analyzed through three interdependent procedures: the driver function, the verification sequence, and the permutation generator.

```
1: while not sorted(list) do
2:     shuffle (list)
3: done
```

*Figure 8. Bogosort Driver Pseudocode. Source: Sanchhaya Education Pvt. Ltd., 2021*

```
1: procedure sorted: {returns true if the array is sorted and  
false otherwise}  
2: for i = 1 to n - 1 do  
3:   if a[i] > a[i + 1] then  
4:     return false  
5:   end if  
6: end for  
7: return true  
8: end procedure
```

*Figure 9. Sorted Procedure Pseudocode. Adapted from: Gruber, Holzer, and Ruepp, 2007*

```
1: procedure shuffle: {permutes the array}  
2: for i = 1 to n - 1 do  
3:   j := rand[i . . n]  
4:   swap a[i] and a[j]  
5: end for  
6: end procedure
```

*Figure 10. Random Permute Pseudocode. Adapted from: Gruber, Holzer, and Ruepp,*

*2007*

The implementation and subsequent analysis of the Bogosort algorithm in this study rely only on one primary data structure: a one-dimensional array. Given that the algorithm has been intentionally unoptimized, and it relies entirely on the basic generate-and-test paradigm, its structural footprint is minimal.

**One-Dimensional Array:** The core sorting operations and evaluations are performed directly within a single list of  $n$  integer elements. This linear sequence holds the sortable integer elements from their original unordered state through every random shuffle.

**In-Place Element Manipulation:** The algorithm executes its permutation phase strictly in-place, swapping elements directly within the original array using their index positions. During

the generation of a new permutation, the shuffle procedure calculates a random target index  $j$  and systematically swaps the values of  $a[i]$  and  $a[j]$  within the boundaries of the original array.

**Absence of Auxiliary Memory Allocation:** Because Bogosort operates directly on the original input array and does not require additional data structures – unlike those conventional, optimized sorting algorithms – it is highly efficient in terms of memory usage, making it an in-place sort.

## VI. Complexity Analysis

Due to its absurdist nature, bogosort presents extreme complexity throughout its rather simple procedure. Following the pseudocode stated for the analysis of its complexity, the time and complexity of the sorting algorithm is as follows:

### Time Complexity

As it's true to any list sorting algorithm, the most optimistic case that an algorithm can successfully execute and sort a list of numerical elements is at  $O(n)$ . In this specific scenario, the arrays would have already been completely sorted in the correct order to achieve a linear time complexity. When a sorted array is given to the algorithm to sort, the only subprocedure that the bogosort algorithm would do is traversing the entirety of the array to check if the elements are sorted. Since the bogosort algorithm uses a pre-checking loop that first identifies if the list is not sorted, the algorithm does not need to implement the shuffling on the first try since it is already verified by the checking subprocess. Therefore, to determine the best-case time complexity, identifying the complexity of the checking subprocess would give us the answer.

Illustrated by figure 9, the checking subprocedure is simply a naive linear array traversal where for the  $i$  value assigned from 1 to the length of the array - 1, the checker would access the  $i$ -th and  $i+1$ th element to see if the latter is larger than the former. If so, the procedure will stop and declare the array as unsorted. Stating that, the time complexity of the checking



would be dependent on the number of elements present in the array. Therefore, the best-case time complexity would be  $O(n)$ .

For the expected or average case time complexity, the analysis would utilize probability and combinatorics to define the odds of returning a sorted algorithm from the shuffling method. As stated earlier, if the unsorted array is pushed through the algorithm and the pre-checking method would return a false value, the shuffling subprocedure of the algorithm will be triggered and the array's elements will be reordered. Once finished, the newly arranged algorithm is again passed into the checker to see if it can be declared as a sorted array. If not, the cycle continues until the array is declared sorted – breaking the loop.

Following Nick Collin's (2006) approach on sorting time complexity for bogobogosort, the expected mathematical expression for the time complexity is:

$$BGS(n) = [Cost\ of\ success\ probability] \cdot [Check() + Shuffle()]$$

$$Cost\ of\ success\ probability = E[X] = \sum_{i=1}^n x_i \cdot P(X = x)$$

Where the cost of the probability of success will be computed as the product of the summation of all k iterations with the probability of success denoted by  $P(X = x)$ . The Check and Shuffling time costs will be added together and added to the probability. This is to illustrate that for every miss from the probability, the costs of the subprocesses will be taken into account.

To determine the probability of a bogosort algorithm's iteration, the total number of permutations that a list of elements would be ordered will be identified as a summation of all possible attempts. Since the bogosort algorithm does not have any save checker or saving mechanism where any significant progress – however significant would be saved in order to reduce the weight of the next iteration according to the given pseudocode, the computation for the probability will use Probability Mass Function of a geometric distribution to determine the total weight of the probability. Geometric Distribution PMF is a phenomenon in which a probabilistic success of an event is of a binary outcome – success or fail. Moreso, the structure of the bogosort algorithm is heavily similar to a geometric distribution in which the probability of success is independent and constant among iterations. The lack of save checking mechanisms makes the probability of success independent among iterations and can be reset after each one. Since a goal of this analysis is to determine how many trials or iterations would it need for the trial to succeed, the geometric distribution PMF is appropriate for the solution.

$$P(X = x) = \sum_{x_1, \dots, x_k = x; x_i \neq 0} \left( \frac{x_1 + \dots + x_k}{x_1, \dots, x_k} \right) \cdot p^x \cdot \left( \frac{p}{q} \right)^{x_1 + \dots + x_k}$$

Following the formula of a generalized geometric distribution as cited from Philippou and Georghiou (1983), the generalized equation will be derived and simplified into a geometric distribution PMF to account for the waiting of one success across iterations rather than counting the number of trials until a k-th success. After derivation the base equation will be as:

$$P(X = x) = p \cdot (1 - p)^{x-1}$$

The terms on the base formula are then substituted to expressions aligning with the probabilistic nature of the algorithm. In the given equation, the  $P(X = x)$  represents the iteration  $x$  where the algorithm succeeds. Following the nature of bogosort, the probability of success on any single shuffle by pure randomness is noted as  $p = \frac{1}{n!}$  where  $n$  is the number of elements in the array. After substituting the values in the formula and further simplification, the expression becomes:

$$P(X = x) = \frac{1}{n!} \cdot \left(1 - \frac{1}{n!}\right)^{x-1}$$

$$P(X = x) = \text{Success at given time} = \frac{1}{n!} \cdot \left(\frac{n! - 1}{n!}\right)^{x-1}$$

Then, the expression for the cost of success at any given time is modified as shown in order to replicate the nature of bogosort in that since there is no save checking mechanism. Therefore, the algorithm can theoretically run forever:

$$E[X] = \sum_{k=1}^{\infty} k \cdot P(X = x)$$

Next term to identify is the time complexity of the shuffle subprocedure. It is probable that the type of shuffling subprocedure will affect the time complexity of the subprocedure. To limit the analysis of the algorithm, the Fisher-Yates shuffling algorithm will be used as the

model for the shuffling subprocedure, specifically a Knuth version where the array traverses from first element to last (Knuth, 1969) (Fisher and Yates, 1963). This aligns with the given pseudocode as shown in figure 10 where in the Fisher-Yates algorithm traverses each element and swaps the two elements randomly picked using the native library. Using the determined time complexity of Fisher-Yates algorithm, the time complexity of the shuffle subprocedure is now  $O(n)$ . The completed expression now becomes:

$$BGS(n) = \sum_{k=1}^{\infty} k \cdot \frac{1}{n!} \cdot \left( \frac{n!-1}{n!} \right)^{x-1} \cdot [O(n) + O(n)]$$

Simplifying the expression, the totality of  $E[X]$  can be compressed as  $O(n!)$ :

$$BGS(n) = \frac{1}{n!} \sum_{k=1}^{\infty} k \cdot \left( \frac{n!-1}{n!} \right)^{x-1} \cdot [O(n) + O(n)]$$

$$\frac{1}{n!} \sum_{k=1}^{\infty} k \cdot r^{k-1} = \frac{1}{(1-r)^2}; r = \frac{n!-1}{n!} :$$

$$\sum_{k=1}^{\infty} k \cdot \frac{1}{n!} \cdot \left( \frac{n!-1}{n!} \right)^{x-1} = \frac{1}{n!} \cdot (n!)^2 = n!$$

$$BGS(n) = n! \cdot [O(n) + O(n)]$$

$$BGS(n) = n! \cdot O(n) = O(n \cdot n!)$$

Therefore, it is defined that the average case for a bogosort implementation is at  $O(n \cdot n!)$ .

Due to the internal structure of the algorithm, it is concluded that the worst-case time complexity of bogosort to be unbounded or  $O(\infty)$ . This is because of the nature of the algorithm where once any element is found to be greater than the next element, the shuffle subprocedure will initiate. Even if the the first  $n-2$  elements of the array are sorted and the last 2 are found not to be, all those  $n-2$  elements will be shuffled nonetheless, resetting any significant progress made and starting all over again.

| Case Scenario | Time Complexity                  |
|---------------|----------------------------------|
| Best Case     | $O(n)$                           |
| Expected Case | $O(n \cdot n!)$                  |
| Worst Case    | $O(\infty)$ ( <i>unbounded</i> ) |

*Figure 11. Summary of Bogosort Time Complexity.*

### **Space Complexity**

Like other conventional array sorting algorithms, the bogosort algorithm only relies on memory for the input of each element in the array. The algorithm does not need to create duplicate arrays or branched subarrays to function. Therefore, the defined space complexity of the algorithm is to be constant  $O(n)$ .

## VII. Implementation Details

The implementation of the Bogosort algorithm was executed using the C++ language. C++ was chosen as the programming language because of its efficient execution and support for structured programming, which are essential for evaluating algorithm performance. Additionally, C++ provides access to the Standard Template Library (STL), which includes built-in utilities for randomization and array manipulation, which are both essential for implementing the generate-and-test behavior of Bogosort.

The implementation follows the iterative structure of Bogosort, wherein the algorithm repeatedly checks if the array is sorted and reshuffles the entire list whenever the ordering condition is not satisfied. The program is composed of three main procedures: a permutation generator (`shuffleArray`), a verification sequence (`isSorted`), and the iterative driver function (`bogosort`). Altogether, these components perform the generate-and-test approach mentioned in the proposed algorithm.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>

using namespace std;

// Helper function to verify if the array is sorted
bool isSorted(const vector<int>& arr) {
    for (size_t i = 1; i < arr.size(); i++) {
        if (arr[i] < arr[i - 1]) {
            return false;
        }
    }
}
```

```

    return true;
}

// Helper function to shuffle the array
void shuffleArray(vector<int>& arr) {
    static random_device rd;
    static mt19937 g(rd()); // setting up a high-quality random number generator
    shuffle(arr.begin(), arr.end(), g);
}

// The raw Bogosort algorithm
void bogosort(vector<int>& arr) {
    while (!isSorted(arr)) {
        shuffleArray(arr);
    }
}

int main() {
    vector<int> data;
    int input, count;

    cout << "How many numbers do you want to sort?: ";
    cin >> count;

    cout << "Enter " << count << " integers: ";
    for(int i = 0; i < count; i++) {
        cin >> input;
        data.push_back(input);
    }

    bogosort(data);

    cout << "Sorted result: ";
    for(int x : data) {
        cout << x << " ";
    }

    return 0;
}

```

*Figure 12. Bogosort in C++ Language Implementation. Adapted from: Gruber, Holzer, and*

The program strictly utilizes the C++ Standard Template Library (STL) to fulfill the requirements of the algorithm:

| Header                         | Purpose of Implementation                                                                                                                                                                     |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;iostream&gt;</code>  | Handled for standard input and output display.                                                                                                                                                |
| <code>&lt;vector&gt;</code>    | Used to create the one-dimensional array required for the in-place element manipulation of $n$ integer elements.                                                                              |
| <code>&lt;algorithm&gt;</code> | Used for the shuffle utility to guarantee uniformly distributed random permutations during the generate-and-test cycles.                                                                      |
| <code>&lt;random&gt;</code>    | Replaces the basic <code>rand()</code> function to provide a robust, non-deterministic Mersenne Twister engine ( <code>std::mt19937</code> ) for the permutation generator/shuffle procedure. |

The use of *mt19937* (Mersenne Twister) as the pseudo-random number generator ensures that the shuffle step produces uniformly distributed permutations, which is the assumption underlying the expected-case complexity analyses of Michael Rosulek and Nick Collins.

The program was developed and executed in a standard C++ coding environment on a regular computer setup. It was run as a console-based application that takes input and displays output through a simple terminal window. No specialized tools or advanced equipment were required, making the implementation simple to operate and reproduce on different computers. This allows the program to be easily tested across different input cases without needing additional software installations. Additionally, the setup ensures that the algorithm's behavior can be consistently monitored during experimentation under the same conditions.



## VIII. Experimental Results / Case Studies

### Overview

The stress tests were conducted across two operating systems: macOS and Windows, to observe whether platform differences introduced any meaningful variation in the behavior of the algorithms. Each platform ran the same suite of test cases under identical timeout conditions of 60 seconds per operation.

**Table 1 - Test Case Implementations**

| Category     |                       |                                          |
|--------------|-----------------------|------------------------------------------|
| Easy cases   | Already Sorted        | {1, 2, 3, 4, 5}                          |
|              | Reverse sorted        | {5, 4, 3, 2, 1}                          |
|              | With duplicates       | {3, 1, 2, 3, 1}                          |
|              | All equal             | {2, 2, 2, 2, 2}                          |
| Medium cases | Random small          | {10, 3, 5, 1, 4}                         |
|              | Random medium         | {34, 23, 122, 9, 45, 67, 12}             |
|              | With negatives        | {3, -1, 2, -5, 0}                        |
|              | Mixed duplicates      | {5, 3, 5, 1, 3}                          |
| Edge cases   | Empty array           | {}                                       |
|              | Single element        | {42}                                     |
|              | Two elements sorted   | {1, 2}                                   |
|              | Two elements reverse  | {2, 1}                                   |
| Hard cases   | Larger random         | {10, 34, 23, 122, 9, 45, 67, 12, 89, 56} |
|              | Many duplicates       | {2, 3, 2, 1, 3, 1, 2, 3}                 |
|              | Negative and positive | {-10, -5, 0, 5, 10}                      |

|                 |                        |                                          |
|-----------------|------------------------|------------------------------------------|
| Absurdist cases | Medium random          | generateRandomVector(100)                |
|                 | Medium with duplicates | generateRandomVectorWithDuplicates(100)  |
|                 | Large random           | generateRandomVector(1000)               |
|                 | Large with duplicates  | generateRandomVectorWithDuplicates(1000) |

---

For absurdist cases, custom functions are used to generate random vector arrays with self-generated elements.

```
vector<int> generateRandomVector(int size, int minVal = 0, int
maxVal = 100) {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dis(minVal, maxVal);

    vector<int> vec;
    vec.reserve(size);
    for (int i = 0; i < size; i++) {
        vec.push_back(dis(gen));
    }
    return vec;
}
```

*Figure 13. Generate Random Vector code in C++ Implementation.*

```
vector<int> generateRandomVectorWithDuplicates(int size, int
minVal = 0, int maxVal = 10) {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dis(minVal, maxVal);

    vector<int> vec;
    vec.reserve(size);
    for (int i = 0; i < size; i++) {
        vec.push_back(dis(gen));
    }
    return vec;
}
```

*Figure 10. Generate Random with Duplicate Element Vector code in C++*

*Implementation.*

The five algorithms tested were Bogosort, Bogobogo Sort, Sleep Sort, Stalin Sort, and Thanos Sort, with Bogosort serving as the primary subject of analysis. All execution times reported represent averages across all four runs. Tests that timed out are marked accordingly and excluded from averages, as an operation with no termination cannot be meaningfully averaged. Average execution times per platform for all test cases are provided in the Appendix.

**Table 2 - Average Execution Time on Small Inputs ( $n \leq 10$ )**

| Test Case               | Input Size | Bogosort | Bogobogo Sort | Sleep Sort | Stalin Sort | Thanos Sort |
|-------------------------|------------|----------|---------------|------------|-------------|-------------|
| Already Sorted          | 5          | 0.000161 | 0.000080      | 0.008838   | 0.000985    | 0.000202    |
| Reverse Sorted          | 5          | 0.000269 | 0.003138      | 0.008647   | 0.000113    | 0.000147    |
| With Duplicates         | 5          | 0.000144 | 0.000731      | 0.006598   | 0.000069    | 0.000140    |
| All Equal               | 5          | 0.000143 | 0.000097      | 0.006067   | 0.000095    | 0.000104    |
| Random Small            | 5          | 0.000204 | 0.001248      | 0.011857   | 0.000104    | 0.000136    |
| Random Medium           | 7          | 0.002464 | 0.110468      | 0.130614*  | 0.000155    | 0.000135    |
| With Negatives          | 5          | 0.000196 | 0.001703      | 0.005898   | 0.000087    | 0.000151    |
| Mixed Duplicates        | 5          | 0.000166 | 0.000460      | 0.008263   | 0.000079    | 0.000154    |
| Empty Array             | 0          | 0.000154 | 0.000147      | 0.000120   | 0.000083    | >60.000     |
| Single Element          | 1          | 0.000143 | 0.000145      | 0.048663   | 0.000094    | 0.000215    |
| Two Elements<br>Sorted  | 2          | 0.000129 | 0.000137      | 0.007503   | 0.000076    | 0.000163    |
| Two Elements<br>Reverse | 2          | 0.000155 | 0.000139      | 0.006898   | 0.000080    | 0.000156    |
| Larger Random           | 10         | 0.738745 | 30.746605     | 0.130500*  | 0.000134    | 0.000181    |
| Many Duplicates         | 8          | 0.000557 | 0.008971      | 0.009236   | 0.000141    | 0.000179    |

|                        |   |          |          |           |          |          |
|------------------------|---|----------|----------|-----------|----------|----------|
| Negative &<br>Positive | 5 | 0.000153 | 0.000132 | 0.014897* | 0.000093 | 0.000139 |
|------------------------|---|----------|----------|-----------|----------|----------|

---

\* *Sleep Sort returned PARTIALLY SORTED on some tests due to algorithm limitations (negatives/concurrency). Thanos Sort timed out on empty arrays due to an infinite loop (known bug).*

The results confirm the theoretical behavior established in the Complexity Analysis, but they do so in concrete, observable terms that pure mathematical derivation cannot fully communicate on its own. The most immediate pattern visible across the table is that Bogosort's execution time is not shaped by the structure of the input in any predictable way. It is shaped entirely by how many random shuffles the algorithm requires before the isSorted verification passes. This is a direct consequence of what the Proposed Solution describes: Bogosort performs no state retention, utilizes no partially sorted information, and resets the entire process completely after each failed shuffle. The algorithm has no memory of what it has already tried.

This characteristic is most legible when comparing Bogosort's already sorted and unsorted results directly. On already-sorted inputs of five elements, Bogosort averages 0.000161 seconds, because the isSorted procedure, as described in Figure 9, performs a single left-to-right traversal and, finding no out-of-order elements, returns true. No call to shuffleArray is ever made. This is the best case described in the complexity analysis: a single  $O(n)$  traversal with no permutation work performed. The moment the input is not pre-sorted, the shuffle subprocedure is triggered, and from that point, the number of iterations required is governed entirely by probability. On a 10-element random input, the average climbs entirely to 0.738745 seconds, over just 5 additional elements. Following the probabilistic framework established in the Complexity Analysis, where each random shuffle has exactly a  $1/n!$  chance of producing the

sorted permutation, the expected number of shuffles required before success is  $n!$ . For  $n=10$ , this figure already exceeds 3.6 million possible permutations. The shuffle procedure used in the implementation produces uniformly distributed permutations, meaning no shuffle is any more likely to land on the sorted arrangement than any other, and previous failures carry no information forward. The algorithm is, by design and by implementation, unable to learn from its mistakes.

When the remaining algorithms are compared with Bogosort, the results indicate clear trade-offs among execution time, correctness, and preservation of the original dataset. Bogobogo Sort demonstrated the weakest practical performance among the completed small-input tests, requiring an average of 30.746605 seconds on the 10-element random case, which was substantially higher than Bogosort's 0.738745 seconds under the same condition. This result is consistent with the expected behavior of Bogobogo Sort, whose recursive structure introduces additional computational overhead beyond that of standard Bogosort.

Sleep Sort completed all small-input test cases within comparatively low execution times. However, several outputs were classified as PARTIALLY SORTED, indicating that shorter times did not always correspond to correct results. Since the algorithm depends on thread timing delays and operating system scheduling rather than direct element comparisons, its reliability is inherently limited. These findings suggest that although Sleep Sort may terminate quickly, it cannot be considered a dependable general-purpose sorting method.

Stalin Sort and Thanos Sort produced the lowest execution times across most valid small-input cases. Stalin Sort frequently completes execution in less than 0.001 seconds because it performs only a single traversal while removing elements that violate ascending order, rather than repositioning them. Thanos Sort likewise achieved low runtimes by repeatedly discarding portions of the array until the remaining elements satisfied the sorted condition. While both algorithms were highly efficient in terms of runtime, neither preserved the full set of original input values.

The comparison demonstrates that execution speed alone is not a sufficient basis for evaluating sorting algorithms. Bogosort and Bogobogo Sort retain all input elements and produce a fully sorted output whenever termination is achieved, but they suffer from severe performance limitations. In contrast, Stalin Sort and Thanos Sort achieve faster execution by sacrificing data completeness, while Sleep Sort sacrifices consistency of results.

**Table 3 - Average Execution Time on Medium and Large Input Tests**

| Test Case              | Input Size | Bogosort | Bogobogo Sort | Sleep Sort | Stalin Sort | Thanos Sort |
|------------------------|------------|----------|---------------|------------|-------------|-------------|
| Medium Random          | 100        | >60.000  | >60.000       | 0.1093962  | 0.00018025  | 0.00027625  |
| Medium with Duplicates | 100        | >60.000  | >60.000       | 0.0829567  | 0.00174     | 0.00026775  |
| Large Random           | 1000       | >60.000  | >60.000       | 0.204434   | 0.0017355   | 0.0034075   |
| Large with Duplicates  | 1000       | >60.000  | >60.000       | 0.1181485  | 0.00066025  | 0.00188475  |

The medium and large input experiments further demonstrate the impracticality of Bogosort and Bogobogo Sort when the problem size increases beyond trivial cases. At both 100 and 1000 elements, neither algorithm completed within the imposed 60-second timeout under random or duplicate-based inputs. This strongly supports the earlier complexity analysis, as the factorial growth of impossible permutations causes the expected runtime to escalate beyond usable limits even at moderate array sizes. In practical terms, these results indicate that Bogosort ceases to function as a realistic sorting method long before reaching large-scale data.

Among the algorithms that completed execution, Sleep Sort showed comparatively stable runtimes, increasing from 0.1093962 seconds on medium random inputs to 0.204434 seconds on large random inputs. However, these outputs were consistently marked as PARTIALLY SORTED, indicating that execution speed alone cannot be treated as evidence of algorithmic correctness. The observed inaccuracies are consistent with the concurrency-dependent design of Sleep Sort, where thread timing and scheduling may produce incomplete or unstable ordering.

Stalin Sort recorded the fastest measured times among algorithms that preserved deterministic behavior, requiring only 0.00018025 seconds for medium random inputs and 0.0017355 seconds for large random inputs. Its low runtime is expected because the algorithm performs a single linear scan while discarding out-of-order elements rather than rearranging them. Likewise, Thanos Sort completed quickly, ranging from 0.00026775 to 0.0034075 seconds, but this efficiency is achieved by repeatedly eliminating portions of the array until the remaining values appear sorted. In both cases, speed is obtained through loss of data rather than genuine sorting.

A secondary visible across the results is that duplicate-heavy inputs often reduce runtime compared with fully random arrays. For example, Sleep Sort decreased from 0.204434 seconds on large random data to 0.1181485 seconds on large duplicate data, while Thanos Sort declined from 0.0034075 to 0.00188475 seconds. Repeated values appear to simplify termination conditions or reduce disorder within the input, allowing faster completion.

Across both operating systems, the results were largely consistent in terms of which algorithms completed, which timed out, and which produced incorrect outputs, with one exception. Sleep Sort was the only algorithm whose correctness differed between platforms, and the difference was stark. On macOS, the algorithm produced correctly sorted output on nearly all small positive-integer inputs across both runs, failing only on inputs containing negative values, which is consistent with what the Literature review already flags as a known limitation of the algorithm. On Windows, however, Sleep Sort returned PARTIALLY SORTED results on the majority of test cases across both runs, including inputs with no negative values whatsoever. This aligns directly with the review's characterization of the algorithm as highly system-dependent, where the reliance on the operating system's scheduler introduces variables that the algorithm itself cannot control. For every other algorithm in the table, no correctness difference between platforms was observed. The variance in Bogosort's execution times, summarized in the Appendix, reflects the independent seeding of the Mersenne Twister on each run rather than any platform difference. This is consistent with the algorithm's generate-and-test structure, which has no platform-dependent components beyond the random number generator.



Taken together, the results support the research objective stated in the Problem definition: to interrogate not just whether an algorithm arrives at a correct answer, but how, and at what cost. Bogosort arrives correctly, completely, and with full data integrity every time it terminates. The cost is measured in seconds, in millions of shuffles, and in the fundamental inability to guarantee when or whether that termination will come. The other algorithms finish fast because they play by different rules, sacrificing data or correctness for speed. The medium and large tests reinforce this central conclusion: runtime alone is insufficient when evaluating algorithms. Some methods finish quickly by cheating, while Bogosort preserves correctness but becomes computationally infeasible as the input size grows. Bogosort plays fair, but fairness does not matter when you might never finish.

## **IX. Discussion**

This section discusses the strengths and weaknesses of Bogosort, the insights gained throughout the development of the study, and the potential edge cases and pitfalls that may affect the validity or reproducibility of the results.

### **Strengths of the Approach**

Despite being widely regarded as one of the least efficient sorting algorithms, Bogosort possesses a small but meaningful set of properties that distinguish it favorably from other esoteric algorithms evaluated in this study. Its most significant strength is output correctness with full data integrity and preservation. Unlike other algorithms, such as Stalin Sort, which removes out-of-order elements, or Thanos Sort, which repeatedly discards half of the array, Bogosort strictly preserves the original dataset. Every value present in the original array is guaranteed to appear in the final ordered sequence with complete data integrity. This property places it in a fundamentally different category from lossy compression algorithms, which may produce a sorted output at the cost of compromising the completeness of the data.

Bogosort is also structurally simple, consisting of only two subprocedures. The Fisher-Yates shuffle to generate the next permutation and the linear traversal to check a sorted order. This operates within a single while loop until it returns true. Its behavior at every step is fully predictable in structure, even if the number of steps required is not.

Related to its simplicity is its memory efficiency. Because Bogosort sorts the input array entirely in-place and does not require the creation of duplicate arrays, subarrays, or auxiliary data

structures, its space complexity is  $O(n)$ —determined solely by the size of the input itself. This stands in contrast to algorithms like Bogobogosort, which recursively copies subsections of the array, and makes Bogosort the algorithm with minimal memory usage among the inefficient algorithms tested.

Finally, Bogosort offers a well-defined and analyzable best case. When the input array is already sorted, the `isSorted` returns true on its first traversal and the shuffle is never called, achieving a runtime of  $O(n)$ . This behavior was confirmed experimentally on already-sorted five-element inputs, and Bogosort averaged 0.000161 seconds. While trivial in practice, it shows that the algorithm's design does avoid unnecessary computation when none is required.

### **Weaknesses of the Approach**

Despite its conceptual simplicity, Bogosort suffers from severe practical limitations that make it unsuitable for real-world use. Its defining weakness lies in its factorial time complexity,  $O(n \cdot n!)$ , where the number of possible permutations grows rapidly as input size increases. For this reason, the expected runtime quickly becomes unfeasible even for relatively small inputs, since the algorithm relies entirely on randomly generating permutations until a sorted arrangement is achieved. In the study, this behavior was observed when Bogosort frequently failed to produce results within the specified execution limits for larger datasets. Its probabilistic nature also introduces large variations in execution time, which means that identical inputs can produce drastically different runtimes depending purely on chance. This makes its performance difficult to measure and unreliable when evaluated using standard averaging techniques.

In addition to its inefficiency, Bogosort demonstrates poor scalability and minimal adaptiveness. While it performs optimally when the input is already sorted, it fails to take advantage of partially sorted data and instead applies the same brute-force randomness in all unsorted cases. As input size increases, performance drops rapidly, making the algorithm impractical beyond small cases. Compared to more efficient sorting algorithms with polynomial time complexity, Bogosort does not scale in any meaningful way. Although it guarantees complete data integrity by preserving all elements in the array, this correctness comes at the cost of extreme computational inefficiency. As such, it is best understood not as a practical algorithm, but as a conceptual tool that highlights the gap between theoretical correctness and real-world usability.

### **Insights Gained During Development**

The study revealed that input structure heavily influences the behavior of inefficient algorithms. Because Bogosort lacks adaptive mechanisms, its execution time only improves if the array is already sorted and bypasses the shuffling phase entirely. Conversely, inputs containing a high density of duplicate values consistently reduced execution times for algorithms like Bogosort, Sleep Sort, and Thanos Sort. Duplicates reduce the total number of unique permutations, which mathematically increases the odds of a probabilistic algorithm randomly landing on a sorted arrangement.

The findings also expose a critical distinction between formal and practical algorithmic correctness. While Stalin Sort and Thanos Sort achieved the fastest execution times, they did so by discarding elements—proving that an algorithm can output a formally ordered sequence while practically failing to preserve data integrity. Lastly, retrieving the original complexity analyses

for Bogobogosort from private 2006 emails demonstrates that rigorous algorithmic reasoning often occurs outside peer-reviewed academia, requiring direct author correspondence to verify.

### **Potential Edge Cases or Pitfalls**

The interpretation and reproducibility of the study's results could be affected by a number of edge cases and methodological limitations. A major issue is the high variability of probabilistic algorithms such as Bogosort and Bogobogosort, where execution times can differ significantly even with identical inputs. This variability makes small sample sizes unreliable and increases the risk that the results do not reflect true expected behavior. Input composition further complicates the analysis: adversarial cases, such as reverse-sorted arrays with unique elements, may exaggerate inefficiency, while already sorted arrays or those containing many duplicates can lead to unusually fast termination. These extremes can distort average runtime measurements and weaken the validity of comparisons.

Additional pitfalls arise from system-level dependencies and implementation details. Algorithms like Sleep Sort are sensitive to operating system scheduling and processor load, which limits consistency across platforms. Although `mt19937` provides high-quality pseudo-randomness, differences in seeding strategies can produce varying execution paths, reducing reproducibility if seeds are not carefully controlled. Furthermore, the use of a fixed 60-second timeout introduces ambiguity by grouping algorithms that are simply slow with those that may not terminate at all, potentially oversimplifying the study's conclusions.

## **X. Conclusion**

### **Summary of Findings**

The experimental results showed that Bogosort's performance is highly dependent on chance rather than input structure, since the algorithm repeatedly shuffles elements randomly until a sorted arrangement is achieved. For already sorted inputs containing five elements, Bogosort demonstrated its fastest performance by completing in approximately 0.000161 seconds. However, execution time increased significantly for unsorted inputs, reaching approximately 0.738745 seconds for a 10-element random array. In medium and large test cases containing 100 and 1000 elements, the algorithm consistently failed to complete within the 60-second timeout limit, demonstrating the impracticality of Bogosort for larger datasets. Bogobogo Sort exhibited even poorer performance due to its recursive and repeated validation process, requiring over 30 seconds for some 10-element inputs and timing out on large tests.

The comparative analysis also revealed major differences in how unconventional sorting algorithms achieve efficiency. Stalin Sort and Thanos Sort consistently produced the fastest execution time across nearly all test cases, often completing in less than 0.001 seconds. However, these algorithms achieved speed by removing elements rather than fully preserving and sorting the original dataset. Stalin Sort eliminated values that violated the sorted order, while Thanos Sort repeatedly discarded random portions of the array until the remaining subset appeared sorted. In contrast, Sleep Sort relied heavily on thread scheduling and operating system timing behavior. Although it completed many tests quickly, it frequently generated partially sorted outputs, especially for larger datasets and certain system environments. The

difference between Windows and macOS further confirms that Sleep Sort's correctness depends strongly on system-level timing precision.

Several additional observations were identified during experimentation. Duplicate-heavy datasets generally reduced execution times for algorithms such as Bogosort and Thanos Sort because fewer unique arrangements needed to be processed before termination conditions were satisfied. The empty-array test case exposed an infinite-loop issue in the Thanos Sort implementation, causing timeout failures. Furthermore, Bogosort displayed highly inconsistent execution times even when identical inputs were used repeatedly, reinforcing the unpredictable nature of randomized algorithms. Among all tested algorithms, Sleep Sort was the only method that showed noticeable correlation variations across operating systems, while the remaining algorithms behaved consistently across platforms.

### **Relevance of the Results**

The results of this study matter because they turn Bogosort's abstract complexity into observable behavior. Its factorial time complexity is not merely a derived expression, it is recorded in timeouts, in millions of shuffles exhausted, in runs that hit the 60-second limit without ever producing a sorted array. These outcomes make the gap between theoretical correctness and practical viability concrete in a way that textbook derivations alone cannot. Bogosort is formally correct, terminates eventually, and preserves every element of the original input. And yet, beyond a handful of elements, it becomes computationally inviable. That combination is precisely what makes it worth studying.

What the comparative results add is a clearer picture of what that inviability actually means. Placed alongside Stalin Sort, Thanos Sort, Sleep Sort, and Bogobogo Sort under identical conditions, Bogosort's failure mode becomes classifiable rather than just observable. It preserves data integrity but cannot guarantee termination at scale. Stalin Sort and Thanos Sort terminate quickly but only by discarding elements, which means they trade completeness for speed. Bogobogo Sort shares Bogosort's commitment to full correctness but compounds its randomness into something strictly slower. Sleep Sort surrenders determinism entirely to system scheduling behavior. Each algorithm fails differently, and Bogosort's particular brand of failure of being honest, complete, and computationally expensive, becomes legible only when the others are placed beside it.

The broader relevance is this: an algorithm can appear to succeed by quietly solving a reduced version of the problem without declaring that reduction. Bogosort exposes that dynamic because it refuses any such shortcut. It is slow precisely because it attempts the full task of generating complete, correctly ordered permutations of the original input, with nothing removed and nothing assumed. That refusal makes it a useful boundary case for what correctness without optimization actually looks like when measured. In the study of algorithm design and analysis, knowing where that boundary lies is as important as knowing how to optimize past it.

### **Future work or possible improvements**

An area for future work is expanding the scale and consistency of the experimental setup. The current study tested Bogosort up to 1000 elements, at which point the algorithm reliably timed out, but that itself is the limit of what the results can say. Larger tests scaled with extended



time allowances, even if termination remains unlikely, would produce more precise data on exactly how quickly the expected runtime escalates relative to input size. Running additional trials per test case would also help address the high variance that makes Bogosort difficult to average meaningfully. A single lucky run can skew results in ways that a larger sample would smooth out. Cross-platform consistency is another consideration worth pursuing, particularly given how differently Sleep Sort behaved between macOS and Windows in this study. Controlling for hardware specifications and operating system scheduling behavior more rigorously across future runs would strengthen the reliability of any comparative conclusions drawn. Future studies could control hardware specifications, operating system scheduling, and thread management more carefully in order to produce more consistent comparisons between platforms. Additional analysis of different randomization methods and implementation techniques may also help determine how these factors influence the behavior of randomized sorting algorithms.

Future researchers may also extend the study by evaluating other unconventional or absurd sorting algorithms not included in the current research, such as Miracle Sort, Quantum Bogosort, and Intelligent Design Sort. Comparing a wider variety of intentionally inefficient algorithms could provide deeper insights into algorithmic behavior, computational impracticality, and the relationship between execution speed, correctness, and data preservation.

## **XI.**

## XII. References

- Al-Kharabsheh, K. S., AlTurani, I. M., AlTurani, A. M. I., & Zanoon, N. I. (2013). Review on sorting algorithms a comparative study. *International Journal of Computer Science and Security (IJCSS)*, 7(3), 120-126.
- Altcademy [Altcademy]. (2023, June 15). Sleep Sort (not practical, but an interesting concept). Altcademy
- Atomisadev. (n.d.). *GitHub - atomisadev/thanos-sort: The most efficient sorting algorithm to ever exist*. GitHub. <https://github.com/atomisadev/thanos-sort>
- Blog.
- <https://www.altcademy.com/blog/sleep-sort-not-practical-but-an-interesting-concept/#problem-statement>
- Chandraseta, R. (2025, April 21). *Esoteric sort algorithms and how to implement them in Python*. Towards Data Science.
- <https://towardsdatascience.com/esoteric-sort-algorithms-and-how-to-implement-them-in-python-96c663fa6ae5/>
- Code, R. (2025, December 11). Sorting algorithms/Sleep sort - Rosetta Code. Rosetta Code.
- [https://rosettacode.org/wiki/Sorting\\_algorithms/Sleep\\_sort](https://rosettacode.org/wiki/Sorting_algorithms/Sleep_sort)
- Constraint Guide - Systematic Search*. (n.d.).
- <https://ktiml.mff.cuni.cz/~bartak/constraints/backtrack.html>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms. MIT press

- CiupercĂ, R. M., Mihai, V. A., & CRIȘAN, D. A. (2023). ALGORITHM EFFICIENCY, A SIDE-BY-SIDE COMPARISON. *Journal of Information Systems & Operations Management*, 17(2).
- Delecluse, A., & Schaus, P. (2024). Black-Box Value Heuristics for Solving Optimization Problems with Constraint Programming (Short Paper). *DROPS (Schloss Dagstuhl – Leibniz Center for Informatics)*. <https://doi.org/10.4230/lipics.cp.2024.36>
- Denning, P. J. (1968). Thrashing. *Fall Joint Computer Conference*, 915.  
<https://doi.org/10.1145/1476589.1476705>
- DM's Esoteric Programming Languages - bogobogosort*. (n.d.).  
<https://www.dangermouse.net/esoteric/bogobogosort.html>
- DM's Esoteric Programming Languages - Dropsort*. (n.d.).  
<https://www.dangermouse.net/esoteric/dropsort.html>
- Fisher, R. A., & Yates, F. (1963). Statistical tables for biological, agricultural and medical research, edited by R.A. Fisher and F. Yates. 6th ed. In Adelaide Research & Scholarship (AR&S) (University of Adelaide). <http://hdl.handle.net/2440/10701>
- GeeksforGeeks. (2025, April 23). *Sleep Sort &ndash; The King of Laziness / Sorting while Sleeping*. GeeksforGeeks.  
<https://www.geeksforgeeks.org/dsa/sleep-sort-king-laziness-sorting-sleeping/>
- GeeksforGeeks. (2021, January 22). Generate and test search. GeeksforGeeks.  
<https://www.geeksforgeeks.org/machine-learning/generate-and-test-search/>
- Group, I. S., Vladlenov, D., Dmytro, K., Maksym, T., & Anastasiia, N. (2023). ANALYSIS OF SORTING ALGORITHM. *MODERN EDUCATION USING THE LATEST TECHNOLOGIES*, 358–361. <https://doi.org/10.46299/isg.p.2023.1.2>

HackerX. (n.d.). *GitHub - HackerX7889/thanos-sort-rs: A perfectly balanced sorting algorithm*.  
GitHub.

<https://github.com/HackerX7889/thanos-sort-rs?tab=readme-ov-file#algorithm-details>

Holzer, M., & Maurer, J. (2018). Selection via the Bogo-Method - more on the analysis of  
perversely awful randomized algorithms. *DROPS (Schloss Dagstuhl – Leibniz Center for  
Informatics)*. <https://doi.org/10.4230/lipics.fun.2018.23>

Ivanchikov, M. M., & Antsuto, A. Y. (2025). Stalin's sort: sorting by removal.

Kiselyov, O., Shan, C. C., Friedman, D. P., & Sabry, A. (2005). Backtracking, interleaving, and  
terminating monad transformers: (functional pearl). *ACM SIGPLAN Notices*,  
40(9), 192-203.

Knuth, D. E. (1969). *The art of computer programming. Volume 2, Seminumerical algorithms*.  
Addison-Wesley.

Lerma, M. A. (2014, June 3). *How inefficient can a sort algorithm be?* arXiv.org.  
<https://arxiv.org/abs/1406.1077v1>

Ma, R., Liang, C. M., Gao, Y., & Yan, F. Y. (2025, October 4). *MetaMuse: Algorithm Generation  
via Creative Ideation*. arXiv.org. <https://arxiv.org/abs/2510.03851>

manifoldlabslimited. (n.d.). *GitHub - manifoldlabslimited/big-oh-no*. GitHub.  
<https://github.com/manifoldlabslimited/big-oh-no>

Merriam-Webster. (n.d.). Bogus. In Merriam-Webster.com dictionary. Retrieved April 5, 2026,  
from <https://www.merriam-webster.com/dictionary/bogus>

Miryanazov, M. (2019). Problem - 1145A. Codeforces.  
<https://codeforces.com/problemset/problem/1145/A>

Pechersky, J. (2023). Prefix-restricted sorting algorithms (Doctoral dissertation, University of North Carolina Wilmington).

Pizzorni. (n.d.). *GitHub - Pizzorni/Bogo: Bogobogosort in Python*. GitHub.

<https://github.com/Pizzorni/Bogo>

*Problem - 1145A - CodeForces.* (n.d.). Codeforces.

<https://codeforces.com/problemset/problem/1145/A>

Sanchhaya Education Pvt. Ltd. (2021, January 22). Generate and test search. GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/generate-and-test-search/>

Shivananda, S. (2023, December 28). *Exploring Sleep Sort*. Medium.

<https://medium.com/@sanjanashivananda07/exploring-sleep-sort-d192d0f11c2d>

Sleep sort - an out of the box sorting algorithm. (2018, September 12). *Rituraj Kaushik*.

<https://riturajkaushik.github.io/2018/09/12/sleep-sort.html>

*Sleep Sort in JavaScript* | <http://t.co/NWJACYK>. (n.d.). Gist.

<https://gist.github.com/thatkookooguy/3b7ce09a9f80a5e6541175104a5d49e9>

*Sorting the Slow Way: an analysis of perversely awful randomized sorting algorithms.* (n.d.).

<http://www.hermann-gruber.com/pdf/fun07-final.html>

*Stalin sort.* (2026, March 9). Sorting Algorithms Visualiser.

<https://www.mintlify.com/DevBoiAgru/SortingAlgorithmsVisualiser/algorithms/stalin-sort>

## XIII. Appendices

### Appendix A. Referenced Algorithms

#### Appendix A.1. Efficient Sorting Algorithms

#### Appendix A.2. Inefficient Sorting Algorithms

```
#include <vector>
#include <iostream>
#include <array>
#include <vector>
#include <iostream>
#define KEY StalinSort
#define float int
#define minor main
#define equal =
#define greater_than >
#define greater_than_or_equal_to >=
#define less_than <
#define less_than_or_equal_to <=
#define increase_by_one ++
#define pointer *
#define getPointer &
#define print std::cout
#define array std::vector
#define enter return
#define repeat for
#define end_of_sentence ;
#define hard_to_detect_error template <typename T>
#define add push_back

//This implementation of stalin sort is easier to read and
understand

hard_to_detect_error
array<T> stalinSort(T pointer arr, size_t count)
{
    array<T> orderedData end_of_sentence
        orderedData.add(arr[0]) end_of_sentence

    unsigned float last equal 0 end_of_sentence
    repeat(unsigned float i equal 1 end_of_sentence i
less_than count end_of_sentence i increase_by_one)
```

```

        {
            if (arr[i] greater_than_or_equal_to
orderedData[last])
            {
                last increase_by_one end_of_sentence
                orderedData.add(arr[i]) end_of_sentence
            }
        }
        enter orderedData end_of_sentence
    }

int minor()
{
    array<float> KEY end_of_sentence
    KEY.add(1) end_of_sentence
    KEY.add(2) end_of_sentence
    KEY.add(5) end_of_sentence
    KEY.add(3) end_of_sentence
    KEY.add(5) end_of_sentence
    KEY.add(7) end_of_sentence

    array<float> ordered equal stalinSort<int>(getPointer
KEY[0], KEY.size());

    for (unsigned float i equal 0 end_of_sentence i less_than
ordered.size() end_of_sentence i increase_by_one)
    {
        std::cout << ordered[i] << "\n";
    }

    enter 0 end_of_sentence
}

```

### **Appendix A.2.1. Stalin Sort in C++ Language Implementation.**

*Reprinted from: <https://github.com/gustavo-depaula/stalin-sort>*

```

#include <vector>
#include <algorithm>
#include <random>
#include <unordered_set>
#include <chrono>

std::vector<int> thanosSort(std::vector<int> arr) {

```

```

std::vector<int> A = arr;
int n = A.size();

unsigned seed =
std::chrono::steady_clock::now().time_since_epoch().count();
std::default_random_engine rng(seed);

while (true) {
    int i = 0;

    while (i < n - 1 && A[i] <= A[i + 1]) {
        i++;
    }

    if (i == n - 1) {
        return std::vector<int>(A.begin(), A.begin() + n);
    }

    int keepCount = n / 2; // floor division for integers
    std::unordered_set<int> indices;

    while (indices.size() < static_cast<size_t>(keepCount))
    {
        std::uniform_int_distribution<int> dist(0, n - 1);
        indices.insert(dist(rng));
    }

    std::vector<int> newArr;
    for (int idx : indices) {
        newArr.push_back(A[idx]);
    }

    A = newArr;
    n = A.size();
}
}

```

**Appendix A.2.2.** Thanos Sort in C++ Language Implementation. *Adapted from:*

*<https://github.com/atomisadev/thanos-sort/tree/main>*

```

#include <chrono>
#include <iostream>

```



```

#include <mutex>
#include <thread>
#include <vector>

std::mutex coutMutex;

void sleepSort(const std::vector<int>& arr) {
    std::vector<std::thread> workers;
    workers.reserve(arr.size());

    for (int num : arr) {
        workers.emplace_back([num]() {
            std::this_thread::sleep_for(std::chrono::milliseconds(num));
            std::lock_guard<std::mutex> lock(coutMutex);
            std::cout << num << ' ';
        });
    }

    for (auto& worker : workers) {
        if (worker.joinable()) {
            worker.join();
        }
    }

    std::cout << std::endl;
}

int main() {
    std::vector<int> arr{34, 23, 122, 9};
    sleepSort(arr);
    return 0;
}

```

**Appendix A.2.3.** Sleep Sort in C++ Language Implementation. *Reprinted from:*

*<https://www.geeksforgeeks.org/dsa/sleep-sort-king-laziness-sorting-sleeping/>*

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <random>

```

```

using namespace std;

void shuffleArray(vector<int>& arr, int n) {
    random_device rd;
    mt19937 g(rd()); // setting up a high-quality random number
generator
    shuffle(arr.begin(), arr.begin() + n, g);
}

void bogobogosort(vector<int>& arr, int n) {
    if (n <= 1) return;

    while (true) {
        // 2. Recursively sort the first n-1 elements
        bogobogosort(arr, n - 1);

        if (arr[n - 1] >= arr[n - 2]) {
            return; // The whole subset is sorted!
        }

        shuffleArray(arr, n);
    }
}

int main() {
    vector<int> data = {3, 1, 2};

    bogobogosort(data, data.size());

    // Print the result
    for(int x : data) {
        cout << x << " ";
    }

    return 0;
}

```

**Appendix A.2.4.** BogoBogoSort in C++ Language Implementation. Adapted From:

*<https://github.com/Pizzorni/Bogo/blob/master/sort.py> and design adapted by Morgan-Mar*

*(2013)*

## Appendix B. Miscellaneous

Nathan Collins <nathan.collins@gmail.com>  
To: David Morgan-Mar <[dmm@dangermouse.net](mailto:dmm@dangermouse.net)>

---

Hi,

Somebody sent me a link to the "intelligent design" sort. That's awesome. Maybe I'm mistaken, but I think I have the complexity of your bogobogosort more or less figured out. You say:

1. Make a copy of the list of numbers.
2. Sort the first  $n-1$  elements of the copy using bogobogosort.
3. Check to see if the  $n$ th element of the sorted copy is greater than the highest element of the first  $n-1$  elements. If so, the copy is now sorted, else randomise the order of the elements of the copy and go to step 2.

4. Check to see if the copy is in the same order as the original list.  
Let  $B(n)$  be the time complexity of bogobogosort on an input of size  $n$ . Making a copy of the list is order  $n$  (assuming the size of the numbers is bounded).

Then  $B(n-1)$  to sort the first  $n-1$  elements.

The chance that the last element in the copy was the largest should be  $1/n$  (assuming the numbers are mostly different on average. Not really compatible with the assumption that the numbers are of bounded size...). So, we probably have to randomize and sort the copy  $n$  times. The check that the last number is largest is constant time (again, bounded number size).

To randomize, you can drop all the numbers on the floor. Dropping them is constant time and it takes order  $n$  to pick them back up and put them back in the computer (assuming you only have two hands). Checking if the two lists are in the same order is order  $n$  (unless we're doing bogobogobogosort, or something like that...).

Then, from the analysis of bogosort, we know we have to try this  $n!$  times, and each time we have to randomize the list again.

OK, so that looks like ( $c_1$  and  $c_2$  are constants)

$$B(n) = c_1 n! (n + (n + n(n + B(n-1) + c_2 + n)))$$

$$\leq c_1 n! (n(2n + B(n-1) + c_2 + 2))$$

$$\leq c_1 n! n^2 B(n-1)$$

for  $n$  large enough that  $B(n-1) \geq 2(n+1) + c_2$ . So, expand recursively:

$$B(n) \leq d c_1 n! (n! (n-1)! \dots 2! 1!) * n! * 2^n$$

for some constant  $d$  that makes up for the fact that our analysis

doesn't make sense for small n.

So, we get  $B(n) = O(c1^n * (n! * (n-1)! * \dots 2! 1!) * n! * 2^n)$  which is  $O(n!^{(n+3)})$  since  $(n-k)!$ ,  $2^n$ , and  $c1^n$  are all  $\leq n!$  for n sufficiently large.

Now  $O(n!^{(n+3)})$  is much better than your guess of  $O(n!^{n!})$ , making bogobogosort even more attractive.

I'll definitely be using bogobogosort until I get ahold of a Demon...  
-nathan

---

Thu, Dec 7, 2006 at 12:41 PM

David Morgan-Mar <dmm@dangermouse.net>  
To: Nathan Collins <[nathan.collins@gmail.com](mailto:nathan.collins@gmail.com)>

---

Thanks for this - it's actually the second analysis of bogobogosort I've received this week. I've added both of them to the page (you come up with different answers!). Now we can have duelling computer scientists... :-)

cheers,  
David

---

Fri, Dec 8, 2006 at 5:00 PM

Nathan Collins <[nathan.collins@gmail.com](mailto:nathan.collins@gmail.com)>  
To: David Morgan-Mar <dmm@dangermouse.net>

---

OK, so now I had to read Mike's analysis carefully, and then reevaluate mine :) I found two minor typos. The first is that you spelled my last name "Colins" instead of "Collins". The second is technical in the part

$$\begin{aligned} BBS(n) &= O(n) + BBS(n-1) + (1-1/n) BBS(n-1) + (1-1/n!) BBS(n) \\ &\text{// multiply by } n! \\ n! * BBS(n) &= O(n*n!) + n! ( BBS(n-1) + 2 BBS(n-1) ) - BBS(n) \\ (n-1)! * BBS(n-1) & \\ &\text{// rearrange} \end{aligned}$$
$$BBS(n) = O(n*n!) + (2n! - (n-1)!)BBS(n-1)$$

In the third line, the term

$$n! ( \text{BBS}(n-1) + 2 \text{BBS}(n-1) )$$

should be

$$n! ( \text{BBS}(n) + 2 \text{BBS}(n-1) )$$

since a  $\text{BBS}(n)$  disappears on that line and reappears on line 5.  
I believe Mike's analysis is sound. It took me a long time to work out the details on the last part

$$\begin{aligned} \text{BBS}(n) &= O(n * n!) + (2n! - (n-1)!) \text{BBS}(n-1) \\ &= O(n * n!) + O(n!) * \text{BBS}(n-1) \\ &= O(n * n!) + O(n * (n!)^2) + \dots + O(n * (n!)^n) \\ &= O(n * (n!)^n), \end{aligned}$$

in particular from line 2 to line 3. In working that out I saw how to improve my estimate in the corresponding part of my argument, namely

$$\begin{aligned} B(n) &= O(c1^n * (n! * (n-1)! * \dots * 2! * 1!) * n! * 2n) \\ &\text{which is } O(n!^{n+3}) \text{ since } (n-k)!, 2n, \text{ and } c1^n \text{ are all } \leq n! \text{ for } n \\ &\text{sufficiently large.} \end{aligned}$$

My argument was pretty poorly written (confusing for me even one day after writing it), so I have rewritten it (motivated by Mike's gradual build up of the formulas and use of big-oh notation from the beginning instead of writing constants) and improved the estimate to  $O(n!^{(n-k)})$  for any fixed  $k$ . The way I improve my estimate would apply equally well to Mike's argument, if he were more precise in the end of his argument. Could you put up my new argument and remove my old one? I think it is much easier to read, and the bound is improved.  
Here goes:

Let  $B(n)$  be the time complexity of bogobogosort on an input of size  $n$ . From the original analysis of bogosort we expect to try times  $n!$  before the random shuffle sorts the list. So,

$$B(n) \leq n! * (\text{Copy}(n) + \text{Check}(n) + \text{Randomize}(n))$$

Making a copy of the list is  $O(n)$  (assuming the size of the numbers is bounded). To randomize, you can drop all the numbers on the floor. Dropping them is  $O(1)$ , and it takes  $O(n)$  to pick them up and put them back in the computer (assuming you only have  $O(1)$  many hands).

To check if the original list is sorted we need to bogobogosort the first  $n-1$  elements of the copy, and then see if the last element of the copy is at least as large as the  $n-1$ st. If the  $n$ th element of the copy isn't maximal we need to randomize the copy again. And, finally, when we have managed to sort the copy, we need to compare it to the original list. So, since the chance that the last element is maximal is at least  $1/n$ , we have

$\text{Check}(n) \leq n * (\text{B}(n-1) + \text{Max}(2) + \text{Randomize}(n)) + \text{Compare}(n)$   
giving

$\text{B}(n) \leq n! * (\text{Copy}(n) + (n * (\text{B}(n-1) + \text{Max}(2) + \text{Randomize}(n)) + \text{Compare}(n)) + \text{Randomize}(n))$

Comparing the two  $n$ -element lists is  $O(n)$ , so we have

$\begin{aligned} \text{B}(n) &\leq n! * (O(n) + (n * (\text{B}(n-1) + O(1) + O(n)) + O(n)) + O(n)) \\ &= n! * (O(n) + n * (\text{B}(n-1) + O(n))) \\ &= n! * (n * (\text{B}(n-1) + O(1) + O(n)) \\ &= n! * (n * (\text{B}(n-1) + n * O(1))) \end{aligned}$

using  $O(n) = n * O(1)$  and  $O(1) + O(n) = O(n)$ . Expanding  $\text{B}(n-1)$  recursively we get

$\begin{aligned} \text{B}(n) &= n! * n^2 * O(1) + n! * n^2 * (n-1)! * (n-1)^2 * O(1) \\ &\quad + n! * n^2 * (n-1)! * (n-1)^2 * (n-2)! * (n-2)^2 * O(1) + \dots \\ &\quad + n! * n^2 * (n-1)! * (n-1)^2 * (n-2)! * (n-2)^2 * \dots * 2! * 2^2 * 1! * 1^2 * O(1) \end{aligned}$

Now, the constant  $O(1)$  is the same for all terms above, and the  $n$ th term bounds the first  $n-1$  terms. So, replacing with the last term throughout, we get

$\begin{aligned} \text{B}(n) &\leq n * n! * n^2 * (n-1)! * (n-1)^2 * (n-2)! * (n-2)^2 * \dots * 2! * 2^2 * 1! * 1^2 * O(1) \\ &= n! * n! * n! * n! * (n-2)! * (n-3)! * \dots * 4! * 3! * O(1) \end{aligned}$

by folding the square terms and the leading  $n$  into the smaller factorials, and absorbing a 2 into the constant. So

$\text{B}(n) = O(n! * n! * n! * n! * (n-2)! * (n-3)! * \dots * (k+1)!)$

where  $k$  is any fixed number and we have absorbed factorials less than  $(k+1)!$  into the constant. Noting that we could bound each  $(n-j)!$  by  $n!$  we can simplify the bound to  $\text{B}(n) = O(n!^{n-k})$ , for any fixed  $k$ .

-nathan

---

Sat, Dec 9, 2006 at 5:06 PM

David Morgan-Mar <dmm@dangermouse.net>  
To: Nathan Collins <[nathan.collins@gmail.com](mailto:nathan.collins@gmail.com)>

---

Nathan Collins wrote:

On 12/8/06, David Morgan-Mar <dmm@dangermouse.net> wrote:

Nathan Collins wrote:

> Hi,

>

> Somebody sent me a link to the "intelligent design" sort. That's  
> awesome. Maybe I'm mistaken, but I think I have the complexity of  
> your bogobogosort more or less figured out.

Thanks for this - it's actually the second analysis of bogobogosort I've received this week. I've added both of them to the page (you come up with different answers!). Now we can have duelling computer scientists... :-)

OK, so now I had to read Mike's analysis carefully, and then reevaluate mine :) I found two minor typos. The first is that you spelled my last name "Colins" instead of "Collins".

Ooos, fixed.

The second is technical in the part

$$\text{BBS}(n) = O(n) + \text{BBS}(n-1) + (1-1/n) \text{BBS}(n-1) + (1-1/n!) \text{BBS}(n)$$

// multiply by n!

$$n! * \text{BBS}(n) = O(n*n!) + n! ( \text{BBS}(n-1) + 2 \text{BBS}(n-1) ) - \text{BBS}(n) \\ (n-1)! * \text{BBS}(n-1)$$

// rearrange

$$\text{BBS}(n) = O(n*n!) + (2n! - (n-1)!) \text{BBS}(n-1)$$

In the third line, the term

$$n! ( \text{BBS}(n-1) + 2 \text{BBS}(n-1) )$$

should be

$$n! ( \text{BBS}(n) + 2 \text{BBS}(n-1) )$$

Also fixed.

My argument was pretty poorly written (confusing for me even one day after writing it), so I have rewritten it (motivated by Mike's gradual build up of the formulas and use of big-oh notation from the beginning instead of writing constants) and improved the estimate to  $O(n!^{(n-k)})$  for any fixed k. The way I improve my estimate would apply equally well to Mike's argument, if he were more precise in the end of his argument. Could you put up my new argument and remove my old one? I think it is much easier to read, and the bound is improved.

No problem, I've replaced your old text with this new one.

The typesetting in HTML got pretty complex there, so let me know if you see an error.

cheers,  
David.

---

Tue, Dec 12, 2006 at 5:37 PM

Nathan Collins <nathan.collins@gmail.com>  
To: David Morgan-Mar <dmm@dangermouse.net>

---

On 12/12/06, David Morgan-Mar <dmm@dangermouse.net> wrote:  
Nathan Collins wrote:

> My argument was pretty poorly written (confusing for me even one day  
> after writing it), so I have rewritten it (motivated by Mike's gradual  
> build up of the formulas and use of big-oh notation from the beginning  
> instead of writing constants) and improved the estimate to  $O(n^{n-k})$   
> for any fixed  $k$ . The way I improve my estimate would apply equally  
> well to Mike's argument, if he were more precise in the end of his  
> argument. Could you put up my new argument and remove my old one? I  
> think it is much easier to read, and the bound is improved.

No problem, I've replaced your old text with this new one.

The typesetting in HTML got pretty complex there, so let me know if you  
see an error.

Thanks. Looks correct. I realize the last part could still be done  
more clearly, but that's my fault, not yours.

Now I can get back to using bogobogosort to solve real world problems...-natha

---

Fri, Dec 15, 2006 at 4:55 AM

**Appendix B.1.** Transcript of Email [Subject: Complexity of Bogobogosort]. Retrieved From:

Nick Collins (2026).

Daedan Ryu Especio Alcantara <daedanryu@gmail.com>  
To: "nathan.collins@gmail.com" <nathan.collins@gmail.com>

---

Good Day, Mr. Collins!

I am Daedan Alcantara, an undergraduate from the Philippines studying Computer Science. I am currently making an analysis paper on bogosort and its performance compared to other existing algorithms such as bogobogosort. I stumbled upon what seems to be your statement quoted from



David Morgan-Mar's Repository named "DM's Esoteric Programming Languages" wherein you studied the time complexity of bogobogosort:

Let  $B(n)$  be the time complexity of bogobogosort on an input of size  $n$ . From the original analysis of bogosort we expect to try  $n!$  times before the random shuffle sorts the list. So,

$$B(n) \leq n! * (\text{Copy}(n) + \text{Check}(n) + \text{Randomize}(n))$$

Making a copy of the list is  $O(n)$  (assuming the size of the numbers is bounded). To randomize, you can drop all the numbers on the floor.

Dropping them is  $O(1)$ , and it takes  $O(n)$  to pick them up and put them back in the computer (assuming you only have  $O(1)$  many hands).

To check if the original list is sorted we need to bogobogosort the first  $n-1$  elements of the copy, and then see if the last element of the copy is at least as large as the  $n-1$ st. If the  $n$ th element of the copy isn't maximal we need to randomize the copy again. And, finally, when we have managed to sort the copy, we need to compare it to the original list. So, since the chance that the last element is maximal is at least  $1/n$ , we have:

$$\text{Check}(n) \leq n * (B(n-1) + \text{Max}(2) + \text{Randomize}(n)) + \text{Compare}(n)$$

giving

$$B(n) \leq n! * (\text{Copy}(n) + (n * (B(n-1) + \text{Max}(2) + \text{Randomize}(n)) + \text{Compare}(n)) + \text{Randomize}(n))$$

Comparing the two  $n$ -element lists is  $O(n)$ , so we have

$$\begin{aligned} B(n) &\leq n! * (O(n) + (n * (B(n-1) + O(1) + O(n)) + O(n)) + O(n)) \\ &= n! * (O(n) + n * (B(n-1) + O(n))) \\ &= n! * (n * (B(n-1) + O(1) + O(n))) \\ &= n! * (n * (B(n-1) + n * O(1))) \end{aligned}$$

using  $O(n) = n * O(1)$  and  $O(1) + O(n) = O(n)$ . Expanding  $B(n-1)$  recursively we get

$$\begin{aligned} B(n) &= n! * n^2 * O(1) + n! * n^2 * (n-1)! * (n-1)^2 * O(1) \\ &\quad + n! * n^2 * (n-1)! * (n-1)^2 * (n-2)! * (n-2)^2 * O(1) + \dots \\ &\quad + n! * n^2 * (n-1)! * (n-1)^2 * (n-2)! * (n-2)^2 * \dots * 2! * 2^2 * 1! * 1^2 * O(1) \end{aligned}$$

Now, the constant  $O(1)$  is the same for all terms above, and the  $n$ th term bounds the first  $n-1$  terms. So, replacing with the last term throughout, we get

$$\begin{aligned} B(n) &\leq \\ &n * n! * n^2 * (n-1)! * (n-1)^2 * (n-2)! * (n-2)^2 * \dots * 2! * 2^2 * 1! * 1^2 * O(1) \\ &= n! * n! * n! * n! * (n-2)! * (n-3)! * \dots * 4! * 3! * O(1) \end{aligned}$$

by folding the square terms and the leading  $n$  into the smaller factorials, and absorbing a 2 into the constant. So

$$B(n) = O(n! * n! * n! * n! * (n-2)! * (n-3)! * \dots * (k+1)!)$$

where  $k$  is any fixed number and we have absorbed factorials less than  $(k+1)!$  into the constant. Noting that we could bound each  $(n-j)!$  by  $n!$  we can simplify the bound to  $B(n) = O(n!^{n-k})$ , for any fixed  $k$ .

Now  $O(n^{n^k})$  is much better than your guess of  $O(n^{n!})$ , making bogobogosort even more attractive.

I looked upon your publications and other studies online regarding this quote and I am experiencing difficulties in finding any sources as Morgan-Mar did not leave any sources to his repository. Attached below is the link for Morgan-Mar's repository on bogobogosort where this quote is reprinted from.

<https://www.dangermouse.net/esoteric/bogobogosort.html>

I'm writing this letter if you could verify these statements written to your name. Furthermore, can I ask for the whole academic paper where this statement came from so that I can add to my current algorithm analysis? I'm looking forward to your reply at your most convenient time. Thank you in advance!

--

Reach Further. Reach Higher.

**Daedan Ryu Especio Alcantara**

Marketing, Programming, Graphics Designer

BS Computer Science, Batangas State University - The National Engineering University

---

Fri, Apr 10, 2026 at 8:33 AM

Nathan Collins <nathan.collins@gmail.com>

To: Daedan Ryu Especio Alcantara <daedanryu@gmail.com>

---

Hello Daedan,

It's funny, I have no memory of writing the analysis that you quoted, but searching my email, I find that it was indeed me, in 2006! There is no paper containing the analysis, it's just something I emailed to the author of that blog post you linked to. I will forward you the original email thread from 2006 where I first wrote down the analysis ...

I'm curious though, how did you figure out I was the Nathan Collins mentioned in the blog post? Did you just Google "Nathan Collins computer science" or something?

---

Fri, Apr 10, 2026 at 11:15 AM

Daedan Ryu Especio Alcantara <daedanryu@gmail.com>

To: Nathan Collins <nathan.collins@gmail.com>

---

Hello Mr. Collins,

Regarding your question, I first looked up your name on academic websites in hopes of finding a paper that mentions studies about algorithms when I found a paper named; "Equational Reasoning about Programs with General Recursion and Call-by-value Semantics". It made sense for me to verify if the Nathan Collins that I saw as the author of the paper was indeed studying Computer Science so I looked up public information and landed on your homepage that was under Portland State University's name.

When I matched the paper with your information, I just took a gamble and hoped that you were the Nathan Collins I'm looking for. To my surprise (and maybe pure luck) I have reached the Nathan Collins!

---

Fri, Apr 10, 2026 at 11:27 AM

Nathan Collins <[nathan.collins@gmail.com](mailto:nathan.collins@gmail.com)>

To: Daedan Ryu Especio Alcantara <[daedanryu@gmail.com](mailto:daedanryu@gmail.com)>

---

Haha, I'm also surprised that you found the right Nathan Collins. There are many people online with my name, but perhaps I'm the only one associated with computer science. Who knows. Anyway, attached is a copy of my original correspondence with the blog post author from 2006 :)



**Complexity of bogobogosort.pdf**  
44K

---

Fri, Apr 10, 2026 at 11:39 AM

Daedan Ryu Especio Alcantara <[daedanryu@gmail.com](mailto:daedanryu@gmail.com)>

To: Nathan Collins <[nathan.collins@gmail.com](mailto:nathan.collins@gmail.com)>

---

Thank you so much! This will greatly help me with what I'm working on. I'll make sure to add your insight on my analysis.

with immense gratitude,  
Daedan

---

Fri, Apr 10, 2026 at 11:45 AM

Nathan Collins <[nathan.collins@gmail.com](mailto:nathan.collins@gmail.com)>

To: Daedan Ryu Especio Alcantara <[daedanryu@gmail.com](mailto:daedanryu@gmail.com)>

---

No problem, glad I was able to help. I'd be curious to see your analysis paper when it's finished.

Sat, Apr 11, 2026 at 2:15 AM

**Appendix B.2.** Transcript of Email [Subject: Inquiry about your statement on bogobogosort].

Daedan Ryu Especio Alcantara <daedanryu@gmail.com>

To: rosulekm@eecs.oregonstate.edu

Good Day, Mr. Rosulek!

I am Daedan Alcantara, an undergraduate from the Philippines studying Computer Science. I am currently making an analysis paper on bogosort and its performance compared to other existing algorithms such as bogobogosort. I stumbled upon what seems to be your statement quoted from David Morgan-Mar's Repository named "DM's Esoteric Programming Languages" wherein you studied the time complexity of bogobogosort:

*Mike Rosulek writes:*

*I believe bogobogosort is  $O(n * (n!)n)$ .*

*Let  $BBS(n)$  be the expected time of bogobogosort on input size  $n$ , and  $SC(n)$  the expected time of the sort checking algorithm on input size  $n$ . Then*

$$BBS(n) = SC(n) + (1 - 1/n!) BBS(n)$$

*// with probability  $1/n!$ , we finish, otherwise repeat  $BBS(n)$*

$$SC(n) = O(n) + BBS(n-1) + (1-1/n)( BBS(n-1) + O(n) )$$

*// with probability  $1/n$ , the  $n$ -th guy is the largest, otherwise must shuffle and re-sort the first  $n-1$  guys*

*Then substituting in for  $SC$  (skipping some simplifications - the  $(1-1/n)*O(n)$  factor gets swallowed by the first  $O(n)$ ):*

$$BBS(n) = O(n) + BBS(n-1) + (1-1/n) BBS(n-1) + (1-1/n!) BBS(n)$$

*// multiply by  $n!$*

$$n! * BBS(n) = O(n*n!) + n! ( BBS(n) + 2 BBS(n-1) ) - BBS(n) - (n-1)!*BBS(n-1)$$

*// rearrange*

$$BBS(n) = O(n*n!) + (2n! - (n-1)!)BBS(n-1)$$

$$= O(n*n!) + O(n!) * BBS(n-1)$$

$$= O(n*n!) + O(n*(n!)^2) + \dots + O(n*(n!)n)$$

$$= O(n*(n!)n)$$

I looked upon your publications and other studies online regarding this quote and I am experiencing difficulties in finding any sources as Morgan-Mar did not leave any sources to his repository.

Attached below is the link for Morgan-Mar's repository on bogobogosort where this quote is

reprinted from.

<https://www.dangermouse.net/esoteric/bogobogosort.html>

I'm writing this letter if you could verify these statements written to your name. Furthermore, can I ask for the whole academic paper where this statement came from so that I can add to my current algorithm analysis? I'm looking forward to your reply at your most convenient time. Thank you!

--

Reach Further. Reach Higher.

**Daedan Ryu Especio Alcantara**

Marketing, Programming, Graphics Designer

BS Computer Science, Batangas State University - The National Engineering University

---

Wed, Apr 8, 2026 at 1:07 PM

Mike Rosulek <rosulekm@eecs.oregonstate.edu>

To: Daedan Ryu Especio Alcantara <daedanryu@gmail.com>

---

Hello,

The text that is quoted is not from any academic paper but from a private email I wrote to this person in 2006! I must have been confident about the analysis at the time. Right now I don't have the energy to re-verify the math.

Mike

---

Wed, Apr 8, 2026 at 1:29 PM

**Appendix B.3.** Transcript of Email [Subject: Inquiry about your statement on bogobogosort].

## Appendix C. Average Execution Time Per Test Category by Operating System

### Appendix C.1. Windows Runs

| Small Input Tests            |            |          |               |            |             |             |
|------------------------------|------------|----------|---------------|------------|-------------|-------------|
| Test Case                    | Input Size | Bogosort | Bogobogo Sort | Sleep Sort | Stalin Sort | Thanos Sort |
| Already Sorted               | 5          | 0.000280 | 0.000141      | 0.016869*  | 0.001925    | 0.000322    |
| Reverse Sorted               | 5          | 0.000444 | 0.004748      | 0.011535*  | 0.000208    | 0.000214    |
| With Duplicates              | 5          | 0.000266 | 0.001008      | 0.011357*  | 0.000148    | 0.000213    |
| All Equal                    | 5          | 0.000266 | 0.000180      | 0.011513   | 0.000154    | 0.000155    |
| Random Small                 | 5          | 0.000334 | 0.001342      | 0.017946*  | 0.000186    | 0.000196    |
| Random Medium                | 7          | 0.004267 | 0.177143      | 0.135867*  | 0.000293    | 0.000201    |
| With Negatives               | 5          | 0.000342 | 0.002547      | 0.008164*  | 0.000161    | 0.000224    |
| Mixed Duplicates             | 5          | 0.000288 | 0.000542      | 0.010226*  | 0.000142    | 0.000233    |
| Empty Array                  | 0          | 0.000231 | 0.000169      | 0.000202   | 0.000151    | >60.000     |
| Single Element               | 1          | 0.000213 | 0.000171      | 0.048615   | 0.000170    | 0.000377    |
| Two Elements<br>Sorted       | 2          | 0.000228 | 0.000151      | 0.012449   | 0.000135    | 0.000297    |
| Two Elements<br>Reverse      | 2          | 0.000240 | 0.000212      | 0.011096   | 0.000140    | 0.000286    |
| Large Random                 | 10         | 0.349793 | 22.291933     | 0.136707*  | 0.000251    | 0.000330    |
| Many Duplicates              | 8          | 0.000720 | 0.010014      | 0.015633*  | 0.000263    | 0.000334    |
| Negative &<br>Positive       | 5          | 0.000281 | 0.000181      | 0.018514*  | 0.000169    | 0.000244    |
| Medium and Large Input Tests |            |          |               |            |             |             |
| Medium Random                | 100        | >60.000  | >60.000       | 0.115911*  | 0.000330    | 0.000488    |
| Medium with<br>Duplicates    | 100        | >60.000  | >60.000       | 0.029442*  | 0.000323    | 0.000480    |



|                           |      |         |         |           |           |           |
|---------------------------|------|---------|---------|-----------|-----------|-----------|
| Medium Random             | 100  | >60.000 | >60.000 | 0.1028820 | 0.0000310 | 0.0000645 |
| Medium with<br>Duplicates | 100  | >60.000 | >60.000 | 0.0116825 | 0.0000255 | 0.0000560 |
| Large Random              | 1000 | >60.000 | >60.000 | 0.1147900 | 0.0001360 | 0.0007835 |
| Large with<br>Duplicates  | 1000 | >60.000 | >60.000 | 0.0272495 | 0.0000840 | 0.0004365 |